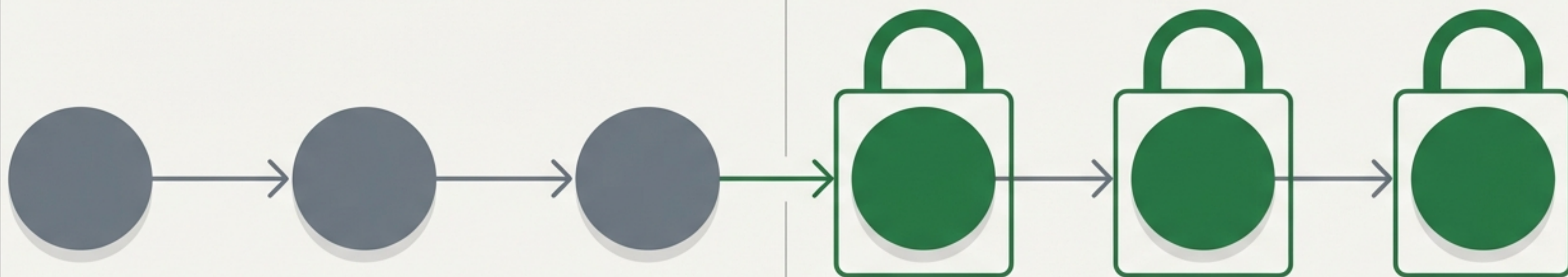


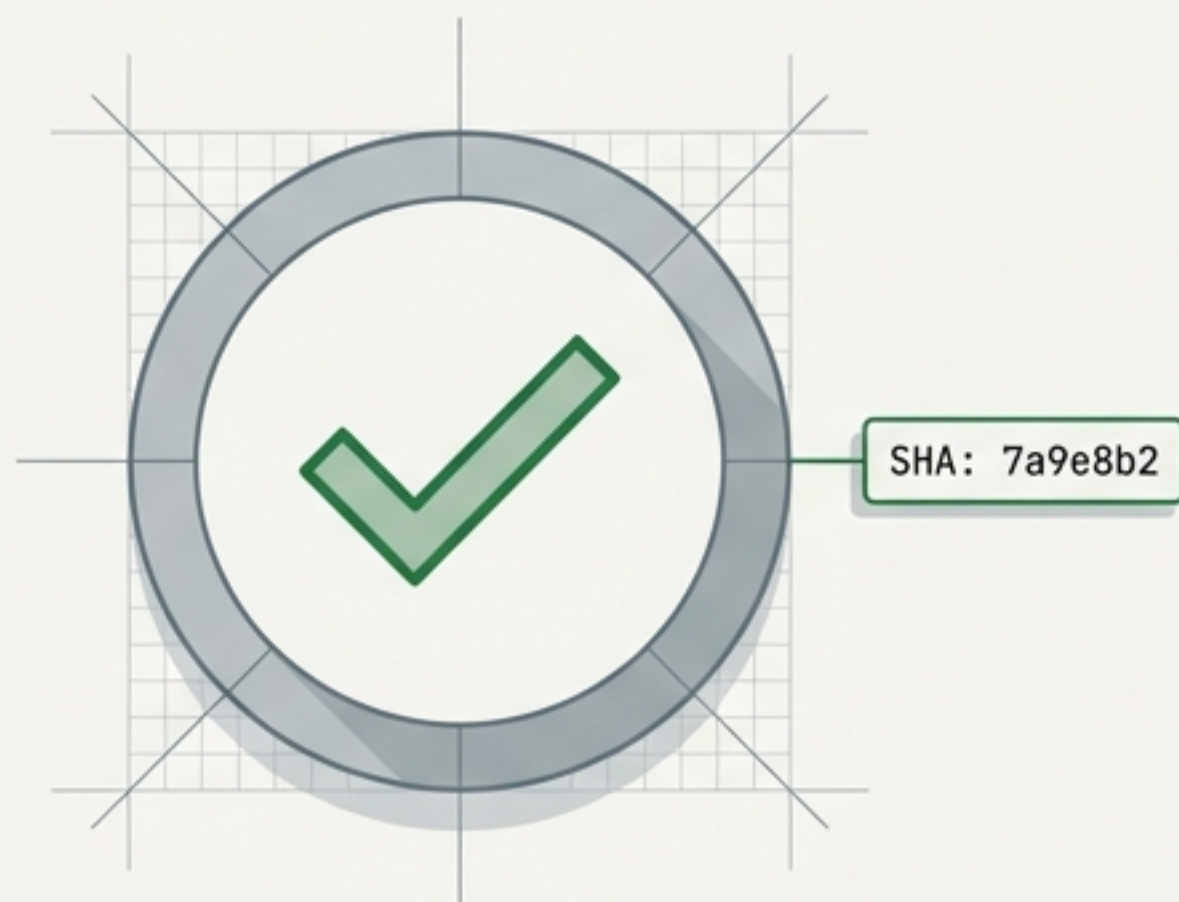
SG Vault

Git with integrated trust.



A version control system preserving the familiar workflow while adding end-to-end encryption, cryptographic identity, and verifiable provenance at the architecture level.

Git secures the history, not the contents



Integrity is solved.

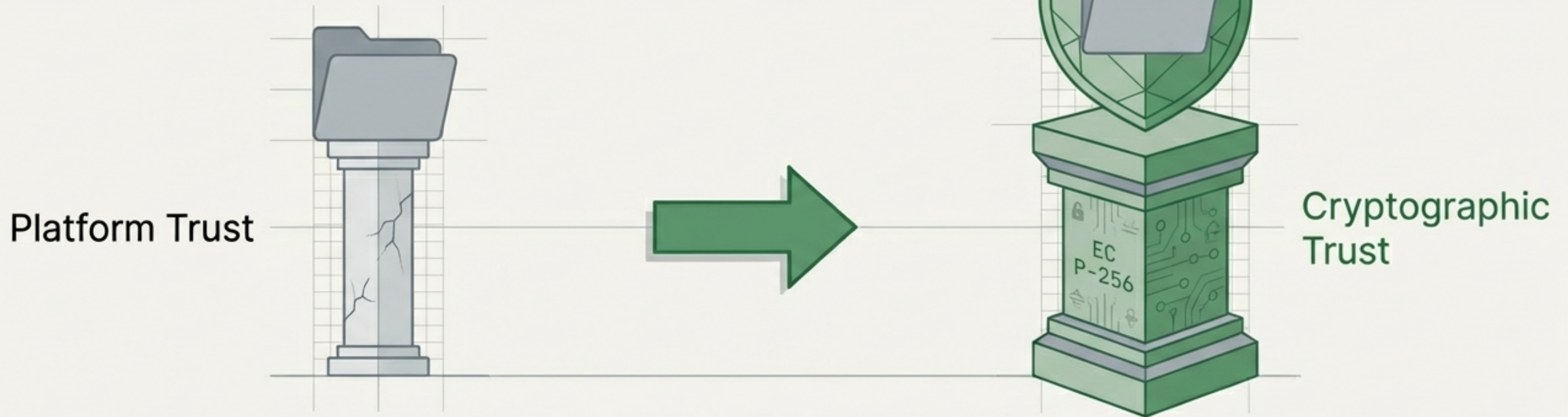
Every object is content-addressed by its hash. The commit graph cannot be silently tampered with.



Confidentiality and identity are missing.

Email addresses are forgeable metadata. Hosting platforms read repository contents in full. You are forced to rely on platform account systems to verify who changed what.

Replacing platform trust with cryptographic trust

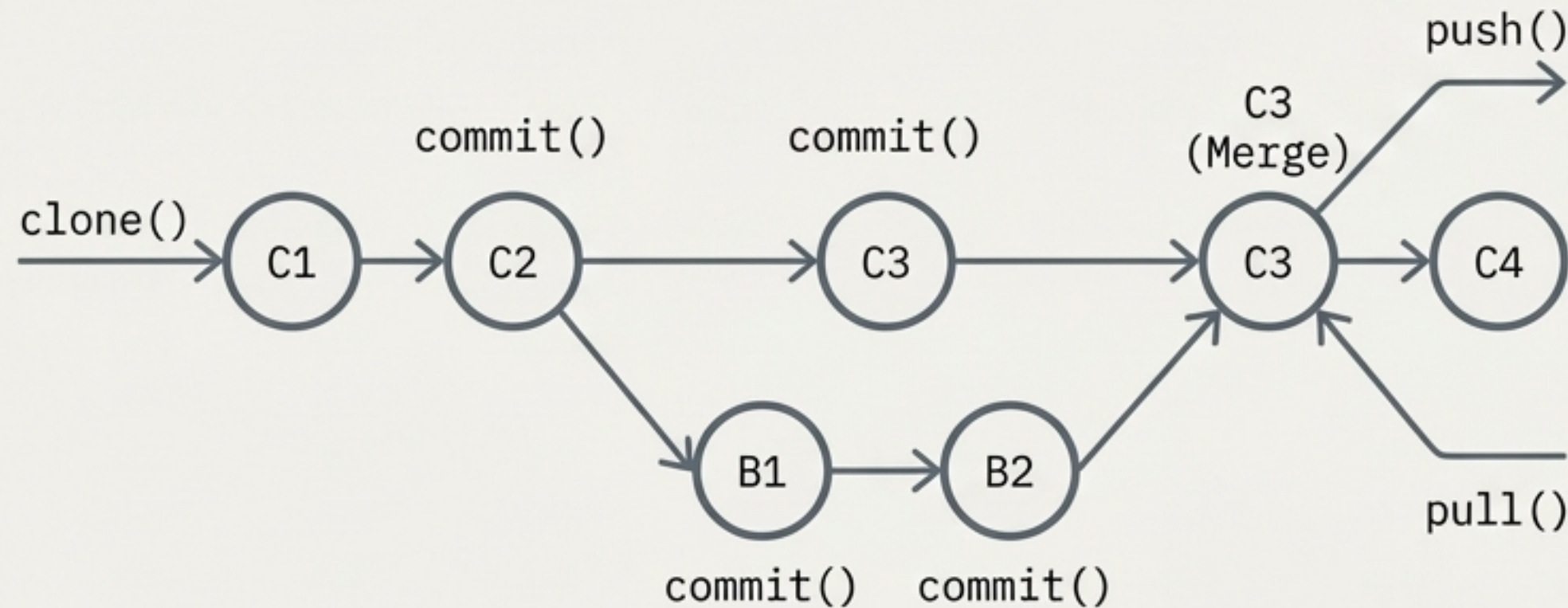


Every file is encrypted before it leaves the client.
Every commit is signed by the branch key that created it.
The server stores only opaque encrypted blobs.

Even if the server is completely compromised, the **integrity and authorship** of every single change remains mathematically verifiable.

The foundation remains identical

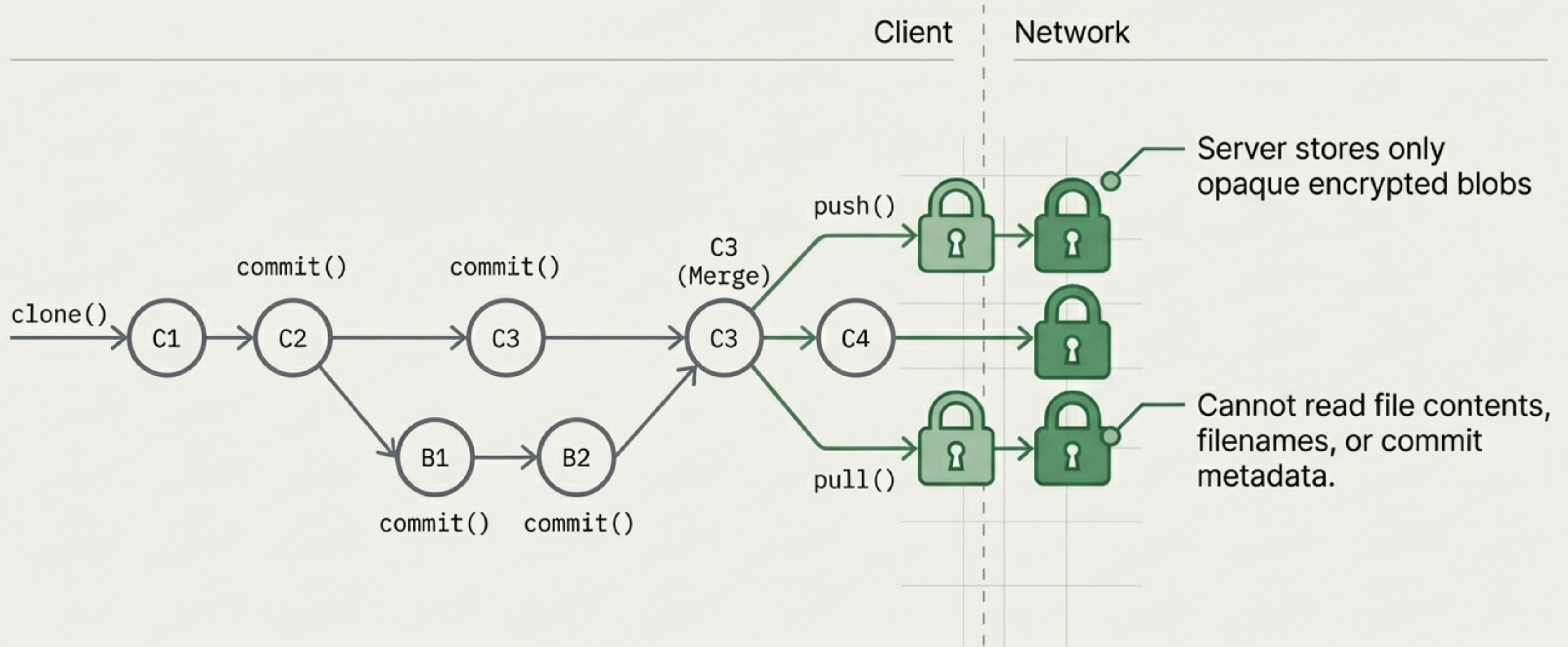
For 90% of what developers do, the workflow and data model remain exactly the same.



- **Commit Graph:** Directed Acyclic Graph (DAG) referencing trees, parents, and metadata.
- **Tree Structure:** One tree object per directory level, capturing complete snapshots.
- **Collaboration:** Diverging branches, independent histories, and standard push/pull mechanics.

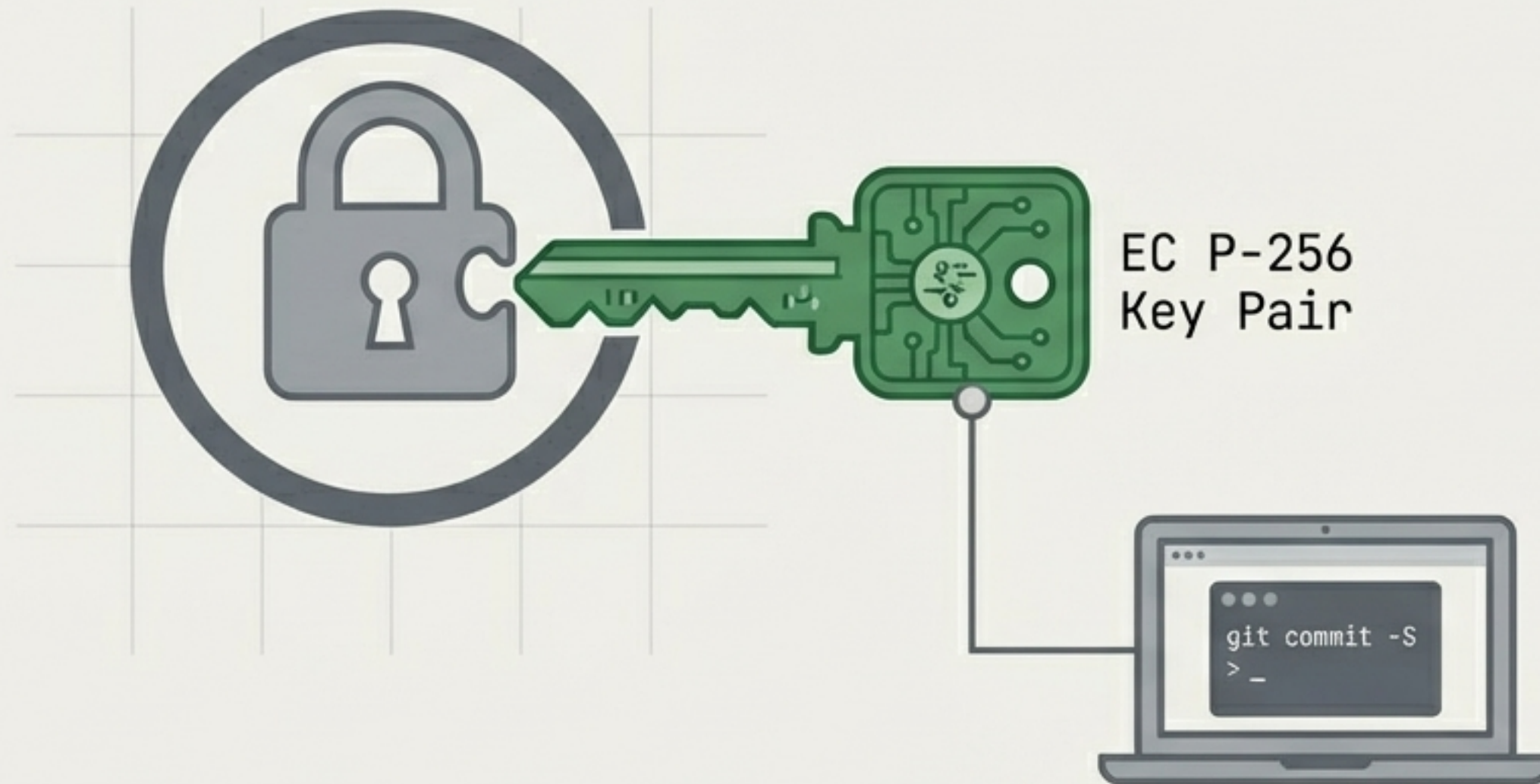
The encryption overlay

All content is end-to-end encrypted on the client before touching the network.



The identity overlay

Cryptographic identity is mandatory.
There are no unsigned commits.

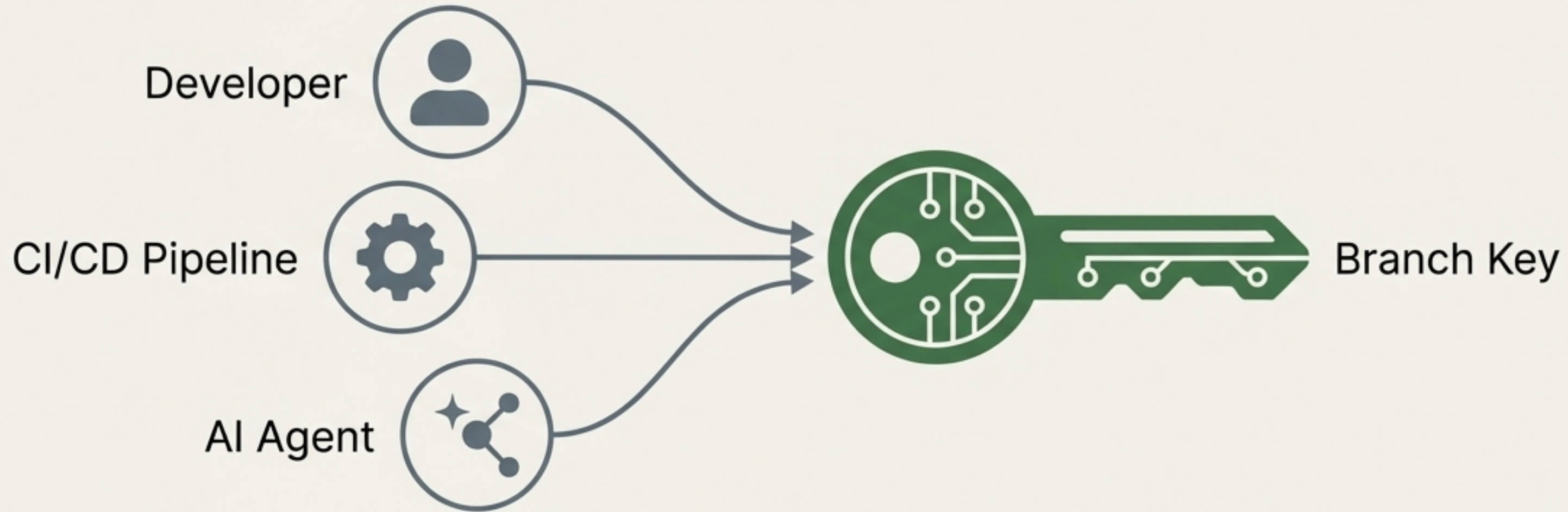


1. Commit
(What changed)
↓
2. Branch Key Signature
(Who changed it)
↓
3. Device / Session
(Where it originated)

You gain cryptographic proof of authorship at the key level, completely bypassing forgeable metadata.

The clone branch is your identity

Every **clone()** automatically generates a new EC P-256 key pair. All your **commits** are signed by your unique branch key.



Because identity is system-agnostic cryptography, it works natively for humans, automated pipelines, and AI tools without requiring specialized service accounts.

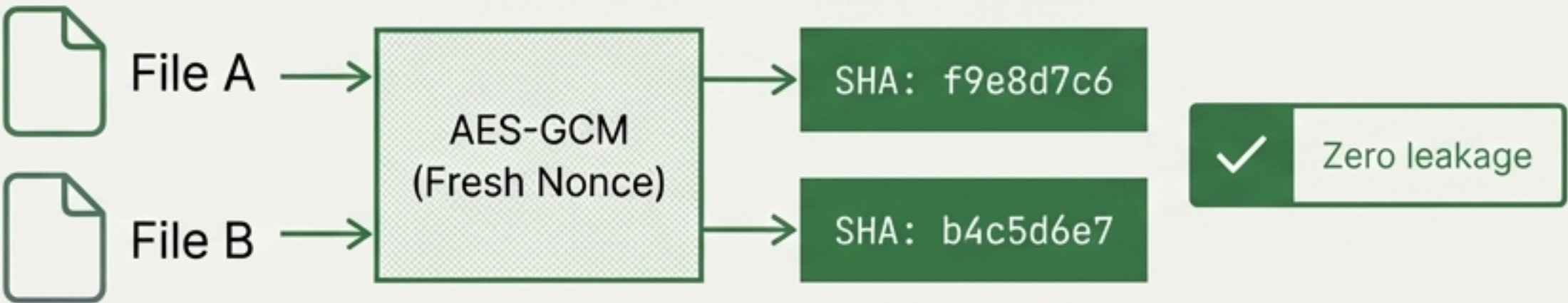
The deduplication trade-off

SG Vault deliberately does not deduplicate across independent encryptions.

Git Pointer Reuse



SG Vault Encryption

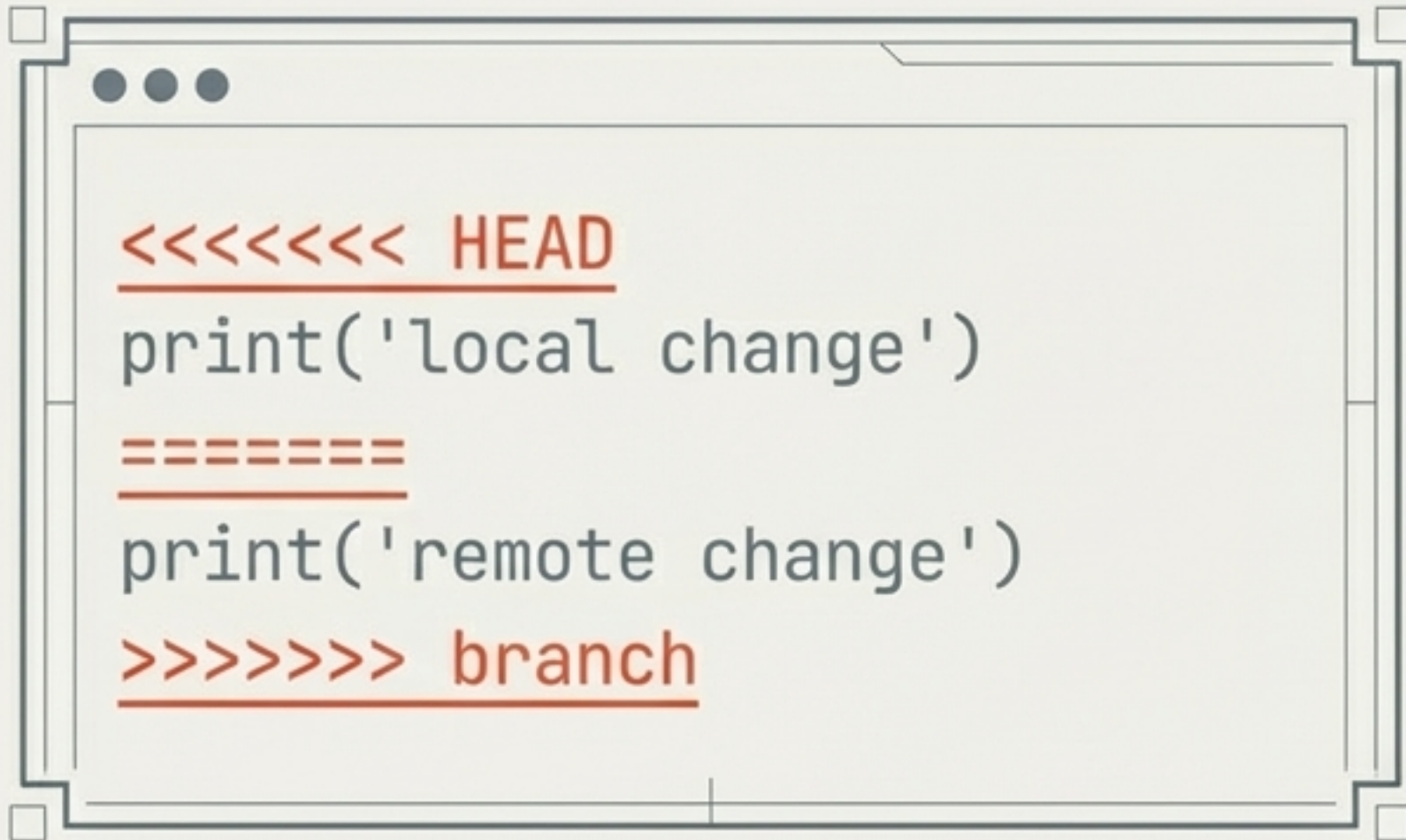


Because AES-GCM uses a fresh nonce each time, identical plaintext encrypted twice produces different ciphertext and different object IDs.

Conclusion: A conscious trade-off—a marginal storage increase in exchange for absolute content confidentiality on the server.

Non-destructive conflict handling

Git

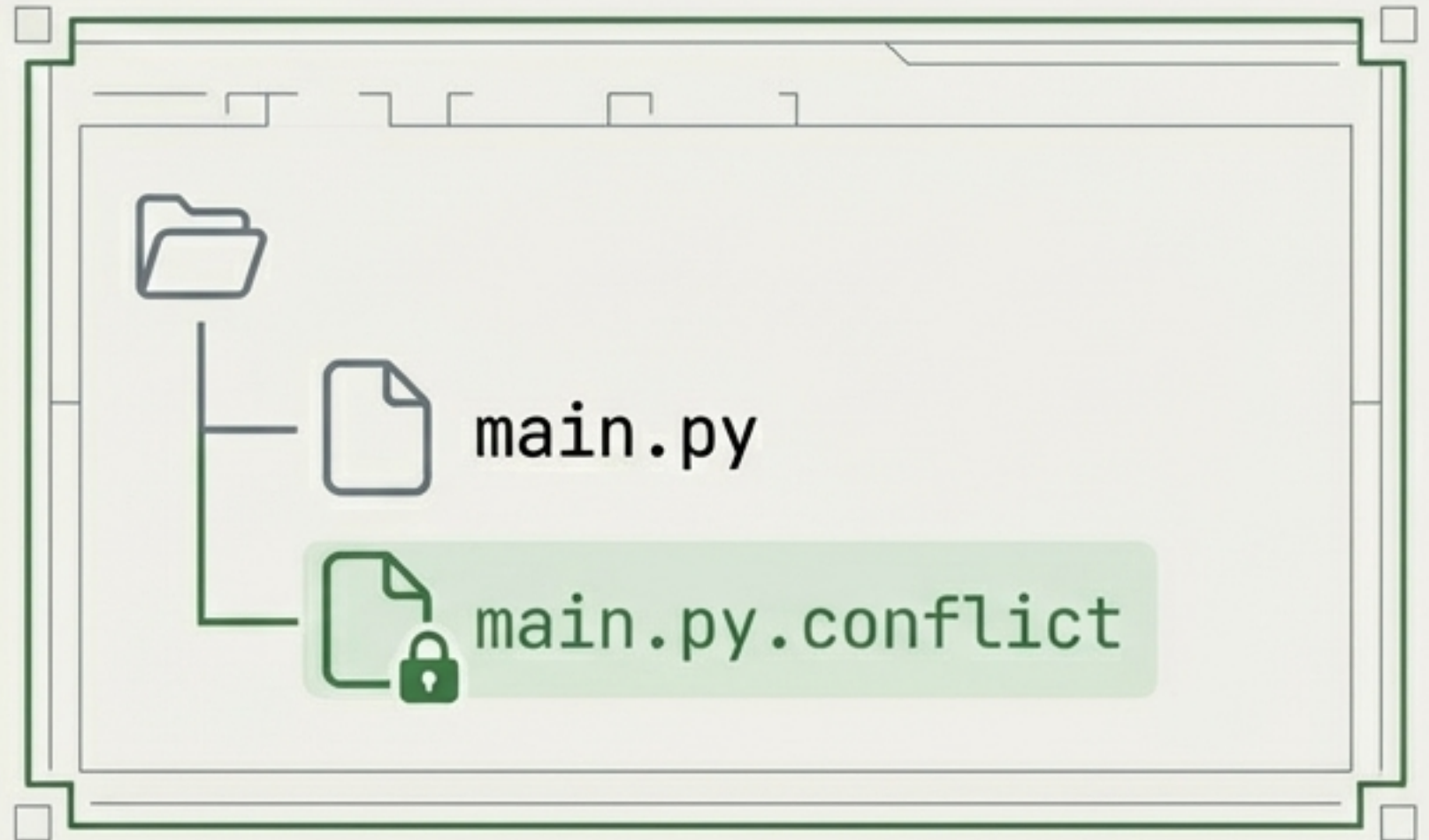
A screenshot of a code editor window with a light gray background. It shows a code diff with merge markers. The markers are red text: six less-than signs followed by 'HEAD' on the first line, six equals signs on the second line, and six greater-than signs followed by 'branch' on the fourth line. The code between the markers is in a standard monospace font:

```
<<<<<< HEAD
print('local change')
=====
print('remote change')
>>>>>> branch
```

Git writes merge markers directly into your files, modifying original content.



SG Vault

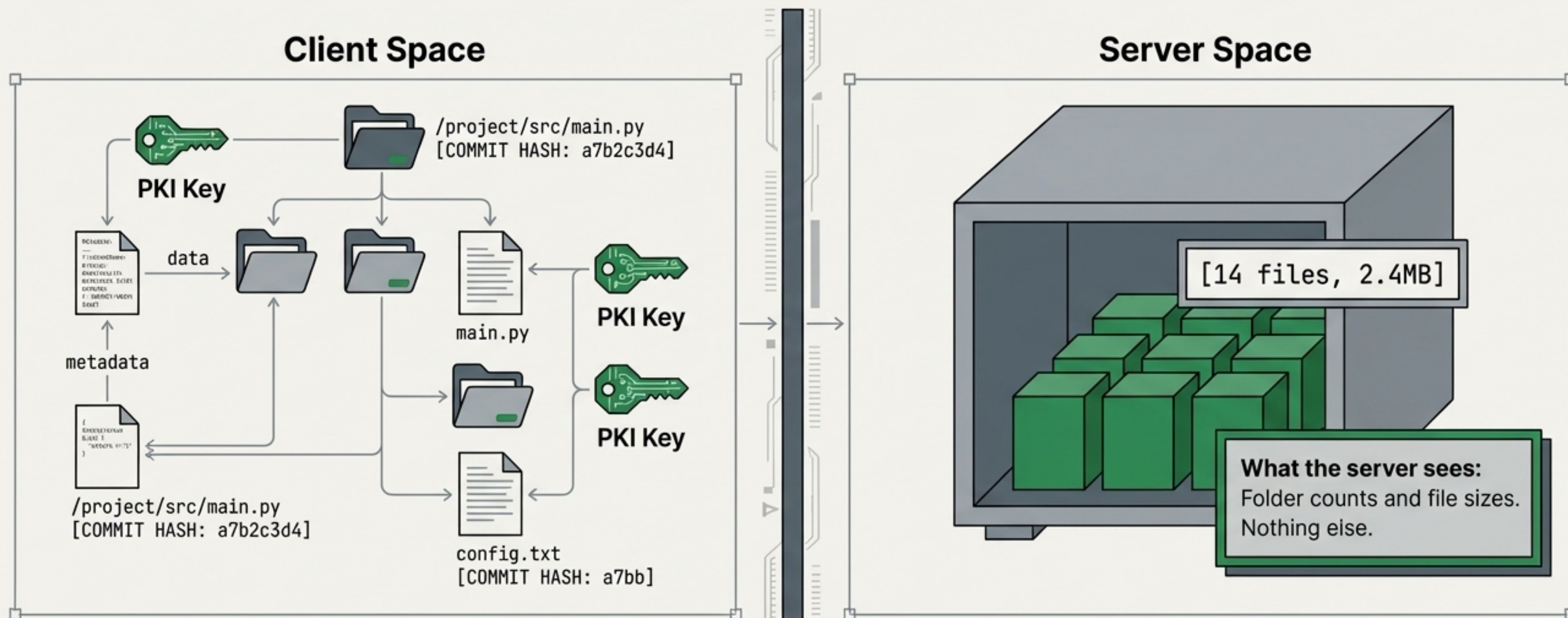


SG Vault never modifies the original file. It creates a **.conflict** file alongside it, preserving your work exactly as you left it.



The agnostic, zero-knowledge backend

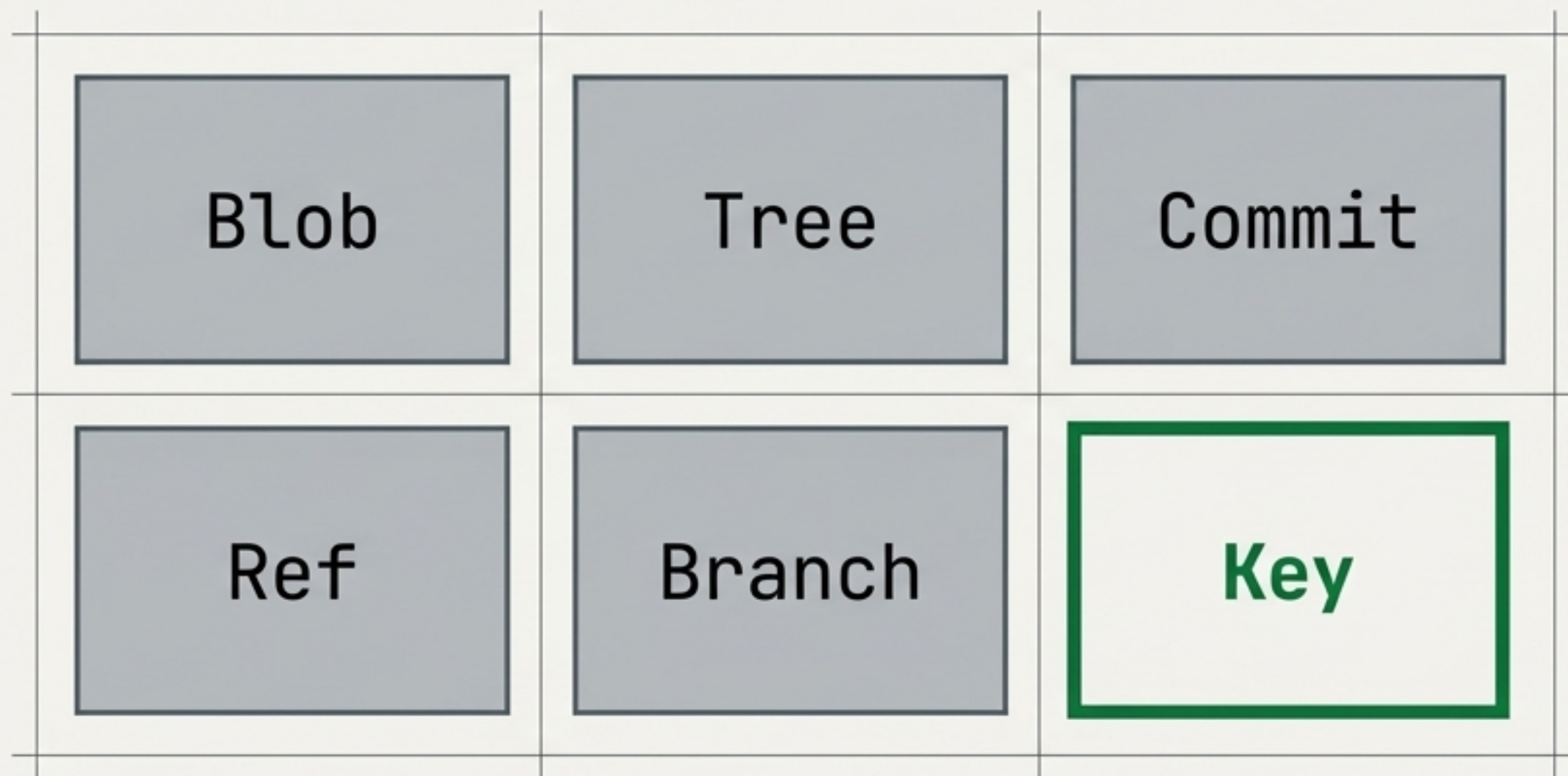
The server is intentionally "dumb". It supports only read, write, delete, and list. No repository logic is required.



Because the backend requires zero logic, vaults can be stored anywhere: S3, local folders, zip archives, or any object store.

Implementation simplicity

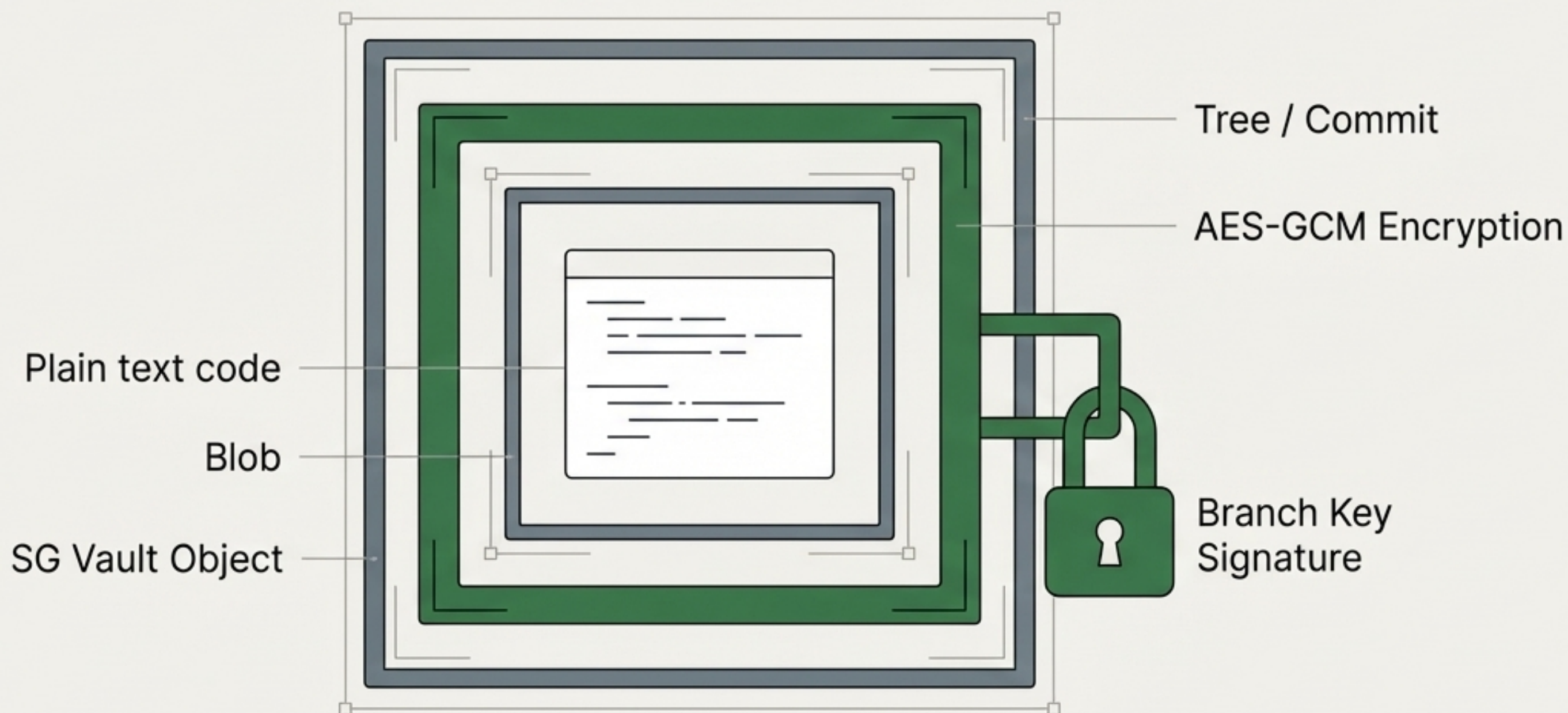
The entire system is built from the exact same fundamental building blocks Git uses, with PKI signing as the only added complexity.



- ✓ Language-agnostic protocol (Python, Go, Rust, Java, Node.js)
- ✓ Straightforward data structures
- ✓ Highly auditable protocol logic

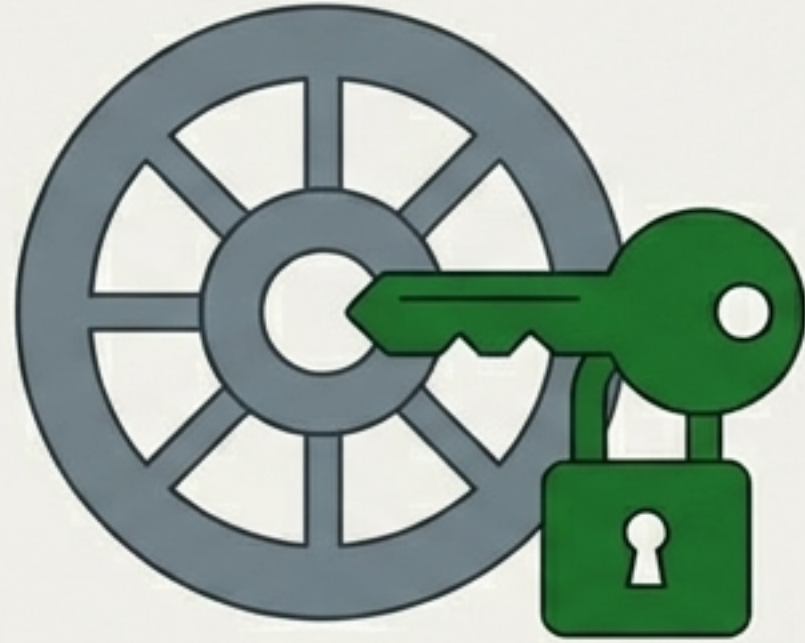
Anatomy of a trust primitive

How standard code becomes a cryptographically secure vault object. Every element is encapsulated, encrypted, referenced, and permanently signed before leaving your machine.



The trust model, compared

Property	Git	Git + GitHub	SG Vault
Content integrity	✓ (SHA hashes)	✓ (SHA hashes)	✓ (SHA hashes)
Content confidentiality	✗	✗	✓ (AES-GCM)
Author identity	✗ (forgeable)	Platform trust	✓ (PKI signatures)
Commit signing	Optional	Optional	Mandatory
Server reads content	N/A	Yes	No
Provenance verification	✗	Account-based	Cryptographic
Compromised server safe?	N/A	No	Yes



Trust the math, not the platform.

SG Vault is what Git would look like if it had been designed with encryption and identity from day one.

Same Workflow • Same Data Model • Uncompromising Cryptography